

P1803R0

packexpr(args, I) -- compile-time friendly pack inspection

Published Proposal, 2019-01-21

Author:

[JeanHeyd Meneide](#)

Audience:

SG17

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1803.html

Abstract

A proposal to refer to a single argument from a variadic pack, piggybacking from the nomenclature and feature set of the Expansion Statements (P1306) paper.

Table of Contents

1 Revision History

1.1 Revision 0 - August 5th, 2019

2 Motivation

2.1 Motivation: Algorithms

2.2 Motivation: a History

3 Design

3.1 Why not `std::get`?

3.2 Syntax

3.2.1 Syntax 0: Keyword Syntax

3.2.2 Syntax 1: Unambiguous Syntax Sugar

3.2.3 Picking a Syntax: Lockouts?

4 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0 - August 5th, 2019

— Initial release.

§ 2. Motivation

Initially, p1306 "Expansion Statements" were slated to go into C++20. With some problems spotted with the syntax and their retargeting to C++23, it has become clear that the syntactic and parsing ambiguities of `for ...` were a bit hairy and convoluted. Still, there is one part of the proposal which calls back to an older paper written by Sean Middleditch and reviewed in 2013, hidden with [\[p1306r1\]](#)'s specification details:

For unexpanded packs, and destructurable objects, the expansion can be trivially implemented in terms of a simple integer index.

In particular, consider the following from [\[p1306r1\]](#):

```

template<typename... Ts>
void f(Ts&&... args) {
    for... (const auto& x : args)
        cout << x << '\n';
}

void foo() {
    f(0, 'a');
}

```

The instantiation of `f` generated from `foo` will have the expansion:

```

{
    {
        const auto& x = /* first element args */;
        cout << x << '\n';
    }
    {
        const auto& x = /* second element in args */;
        cout << x << '\n';
    }
}

```

Note specifically, the following expression in the paper:

```

const auto& x = /* first element args */;

```

This proposal is for the thing that goes in the comment here that the paper hand-waves as "`get-expr\(I\)", where I is the I^{th} element of the pack. As this functionality already needs to be generated by a feature that everyone agrees they want (modulo changes during the C++23 design phase), it seems timely that we standardize a feature that was previously proposed and told to come back with a (non-polled but noted in the minutes) strong consensus for a language feature.`

§ 2.1. Motivation: Algorithms

The largest and most striking problem of packs is the inability to match patterns of input or conduct efficient order-dependent linear processing on the contents of said pack without resorting to recursive operations or worse.

Consider, for example, a function template which is used to process 2 elements at a time from its variadic parameter expressions:

```
void two_at_a_time() {
    // base case
}

template <typename Arg0, typename Arg1, typename... Args>
void two_at_a_time (Arg0&&... arg0, Arg1&&... arg1, Args&&... args) {
    // process arg0, arg1
    two_at_a_time(std::forward<Args>(args)...);
}
```

Unfortunately, `template for` can only offer one-way, one-at-a-time, linear processing of its elements. There is no way to get out 2 elements at a time from a parameter pack with the proposed language feature:

```
template <typename... Args>
void two_at_a_time (Args&&... args) {
    template for (auto& arg : args) {
        // Uhhh...
    }
}
```

Yet still, a `std::initializer_list` of a `std::pair` of types does not work because the list contains heterogeneous types that cannot be fixed before-time. Or worse, one would need to instantiate a tuple, and then use `std::index_sequence` tricks to index into the tuple in a custom manner. `std::tuple` might actually be the worst of the problem here: it is its own Russian Doll of template instantiations that does not work well with the compiler's execution time or memory consumption at all. Most flagrant of the implementation's abuses here is the VC++ Standard Library's implementation of `std::tuple`, which is a recursively-self-deriving structure which bites off one element of the type list and then delegates all further elements down to its bases, until it reaches the empty tuple.

§ 2.2. Motivation: a History

Previously, Sean Middleditch wrote [\[N3761\]](#) back in 2013. It was a library extension to do much the same as is going to be proposed in this paper. The feature was discussed extensively during the Chicago, Illinois 2013 C++ Standards Meeting to standardize exactly this functionality, except as 2

library-based functions. The feedback he was given at the time was that there was strong encouragement to release this not as a library feature, but as a language feature for various reasons.

- Compile-time complexity problems with a library feature;
- function template instantiation costs even with the use of a built-in or intrinsic internally inside of the library function;
- and, compiler memory consumption is *enormous*.

It was perhaps thought that a variadic pack of e.g. `Args... args` could be given the random-access pack indexing of `args...[I]`, where `I` is a constant evaluated integer value. The problem is that this conflicts with an [existing declaration that is entirely valid code](#) (Clang accepts, GCC and VC++ reject, but Clang's interpretation seems valid as per the C++ grammar).

The paper was not pursued, mostly because the original author has not had the time to do so. Further papers from Bill Seymour and Stephan T. Lavavej -- [\[N4144\]](#) -- as well as Daveed Vandevoorde -- [\[N4235\]](#) -- were presented a year later. Both were heavily encouraged for future work towards a language direction and to bring additional examples. Both [\[N3761\]](#)'s discussion and [\[N4144\]](#)'s discussion came to the same conclusion: template metaprogramming proposals "ultimately compensate for shortcomings of the core language".

Daveed Vandevoorde's [\[N4235\]](#) is the language variant of all of the above. In it, he avoids the earlier syntactic ambiguities present with `args...[I]`, and instead chose the syntax `args.[I]`. This was both unambiguous and rather unique, and could be applied without breakage for both types and values. Evolution Working Group polls at the time were largely supportive of this effort:

Proceed with proposed syntax (`args.[index]`)?

SF	F	N	A	SA
5	11	8	3	0

Choose some other syntax (those who are OK with syntax should vote No/Against)?

SF	F	N	A	SA
3	1	16	2	1

Unfortunately, this is where the trail goes absolutely cold. It is unclear whether early Reflection discussion dominated the arena or whether refocused efforts left the papers largely untouched. As has become a modus-operandi of this paper's author, this paper is being picked up due to widespread

indication that something to access packs directly has been requested by developers from multiple software engineering-influenced disciplines. The author has also encountered the need for this [when combating compile time issues and code bloat](#) in a heavily templated library, as have many others. `std::tuple` is one of the greatest offenders of compile-time, alongside [common functions such as `std::move` and `std::forward`](#). While we cannot solve all of the compile-time issues in generic libraries, this proposal represents a useful opportunity to prevent additional compile time bloat as well as make it easier to express simple algorithms.

§ 3. Design

The feature is designed as a language feature, rather than a library feature. Working with the Schrödinger-like entity that are variadic packs in the C++ language is a tedious and computationally expensive (at compile-time) chore. They can contain one (or more) values or types, and cannot be inspected unless something is instantiated over those values or types, templated or not. This means that generic code has often had to perform recursive function template and class template instantiation to process types and values, or worse. Therefore, this feature focuses on providing random-access, constant evaluated integral expressions as the means of accessing pack values and pack types.

§ 3.1. Why not `std::get`?

`std::get` can be ambiguous and dangerous to overload for this purpose. Consider the following:

```
template <typename... Args>
void f (Args&& ... args ) {
    decltype(auto) expr0 = std::get<0>(args...);
}

int main () {
    std::tuple<int> thing0{0};
    foo(thing0);
}
```

Here, `expr0` is not `std::tuple<int>` -- it is `int`, due to the previous overloads to retrieve a value out of a single tuple. This makes it very important to separate what this low-level pack access tool means from any generic tuple-unwrapping facility proposed and in the standard to date, lest we risk severe collisions in the domain space.

§ 3.2. Syntax

There are two potential syntaxes for this feature, a keyword and built-in grammar sugar.

§ 3.2.1. Syntax 0: Keyword Syntax

This first syntax uses a new keyword to retrieve the value of a pack, at the specified index. It is used as follows:

```
template <typename... Args>
void foo(Args&&... args) {
    decltype(auto) x = packexpr(args, 1);
}
```

It results in an error if the index is greater than `sizeof... (Args)`:

```
template <typename... Args>
void foo(Args&&... args) {
    decltype(auto) x = packexpr(args, 1);
}

int main () {
    foo("hi"); // produces error from within above function
    return 0;
}
```

There is no direct expression for retrieving the type out of a pack, because that can be done by just using `decltype()` around `packexpr(args, index)`.

The potential names for the language keyword could be as follows:

- `pack_expr`, `packexpr` (~200 unique hits in preliminary code searches)
- `get_pack_expr`, `getpackexpr` (0 unique hits in preliminary code searches)
- `pack_get_expr`, `packgetexpr` (0 unique hits in preliminary code searches)
- `static_pack_get` (0 unique hits in preliminary code searches)
- `pack_get` (~2500+ unique hits in preliminary code searches, including usages with Boost.HOF)

This proposal, despite the collisions, prefers `packexpr` for being one of the best words to describe this functionality so far. However, there is pre-existing code that takes the names `packexpr` and `pack_expr`. The author would not consider it a large breakage, but a breakage nonetheless: further name bikeshedding should be done to find a name that is still just as cromulent and descriptive that falls in line with `decltype()` and friends.

§ 3.2.2. Syntax 1: Unambiguous Syntax Sugar

This is the second syntax option. It was the one previously proposed and voted heavily in favor of in Evolution Working Group, as demonstrated in the [polls above](#).

The functionality is exactly the same as [Syntax 0](#), just that it comes with the form `args.T[index]` instead. The paper originally proposing this feature wanted to do pack selection and separation as well, allowing arbitrary packs and type names to be reconstituted from the original pack using a syntax somewhat similar to `args.<index0, index1, ..., indexN>`. We do not propose this syntax at this time, wanting to focus on just the core viable use case that can reduce compilation times for users.

```
template <typename... Args>
void foo(Args&&... args) {
    decltype(auto) x = args.[1];
}
```

It results in an error if the index is greater than `sizeof...(Args)`:

```
template <typename... Args>
void foo(Args&&... args) {
    decltype(auto) x = args.[1];
}

int main () {
    foo("hi"); // produces error from within above function
    return 0;
}
```

§ 3.2.3. Picking a Syntax: Lockouts?

Neither syntax prevents future developing of the idea in either keyword form or syntax sugar form. Slices can still be developed for both of these.

§ 4. Acknowledgements

Thanks to Sean Middleditch for the original proposal and helping me track down the history of it and informing me of its fate. Thanks to Richard Smith for helping show me why the `args... [index]` syntax was ambiguous. A special mention to Barry Revzin, who was able to help track down the rest of this paper's history.

§ References

§ Informative References

[N3761]

Sean Middleditch. [Proposing type_at<>](https://wg21.link/n3761). 29 August 2013. URL: <https://wg21.link/n3761>

[N4144]

B. Seymour, S. Lavavej. [Searching and Manipulation of Parameter Packs](https://wg21.link/n4144). 11 September 2014. URL: <https://wg21.link/n4144>

[N4235]

Daveed Vandevoorde. [Selecting from Parameter Packs](https://wg21.link/n4235). 10 October 2014. URL: <https://wg21.link/n4235>

[P1306R1]

Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. [Expansion statements](https://wg21.link/p1306r1). 21 January 2019. URL: <https://wg21.link/p1306r1>